

utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

Politecnico di Torino

Corso di Programmazione di Sistema / API Programming

Esame Completo — Appello del 3 Luglio 2026

Docente: Programmazione di Sistema — **Studente:** Nicola Beccaceci (s360641)

3 Luglio 2026

Parte I: Domande Teoriche (Valutazione: 9,0 punti)

Domanda 1 (Teoria Lifetimes & Borrow Checker) — 3,0 punti

Si consideri il seguente programma scritto in linguaggio Rust:

```
1 struct Parser<'a> {
2     text: &'a str,
3     pos: usize,
4 }
5
6 impl<'a> Parser<'a> {
7     fn new(text: &'a str) -> Parser<'a> {
8         Parser { text, pos: 0 }
9     }
10
11     fn rest(&self) -> &str {
12         &self.text[self.pos..]
13     }
14 }
15
16 fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
17     if x.len() > y.len() { x } else { y }
18 }
19
20 fn main() {
21     let s1 = String::from("un testo abbastanza lungo");
22     let risultato;
23     {
24         let s2 = String::from("breve");
25         risultato = longest(&s1, &s2);
26         println!("Piu lungo: {}", risultato);
27     }
28
29     let p = Parser::new(&s1);
30     println!("Resto: {}", p.rest());
31
32     // Domanda B: si consideri di decommentare la riga seguente
33     // println!("Ancora: {}", risultato);
34 }
```

Quesiti:

1. Che cosa significa l'annotazione `'a` nella funzione `longest`? Quale relazione stabilisce tra i parametri di input e il valore di ritorno? (0,5 pt)

2. Se si decommenta la riga indicata (`println!("Ancora: {}", risultato);`), il programma compila? Motivare la risposta con l'analisi del Borrow Checker. (1,0 pt)
3. Nel metodo `rest` non compare alcuna annotazione esplicita sul tipo di ritorno: perché il codice è comunque valido? (0,5 pt)
4. Perché la struct `Parser` deve dichiarare il parametro generico `<'a>`? Cosa accadrebbe senza? (0,5 pt)
5. Si potrebbe riscrivere `longest` con due lifetime distinti: `fn longest<'a, 'b>(x: &'a str, y: &'b str) -> &'a str`? In quali casi compilerebbe? (0,5 pt)

Domanda 2 (Teoria Concorrenza & MPSC Channels) — 3,0 punti

Si consideri il seguente programma Rust basato su thread e canali `std::sync::mpsc`:

```

1 use std::sync::mpsc;
2 use std::thread;
3
4 fn main() {
5     let (tx1, rx1) = mpsc::channel::<u32>(); // canale 1
6     let (tx2, rx2) = mpsc::channel::<u32>(); // canale 2
7
8     let mut producers = Vec::new();
9     for id in 0u32..2 {
10         let tx1c = tx1.clone();
11         let p = thread::spawn(move || {
12             for k in 1..=2 {
13                 tx1c.send(id + k).unwrap();
14             }
15         });
16         producers.push(p);
17     }
18
19     let transformer = thread::spawn(move || {
20         for v in rx1 {
21             tx2.send(v * v).unwrap();
22         }
23     });
24
25     let consumer = thread::spawn(move || {
26         let mut somma = 0u32;
27         for v in rx2 {
28             somma += v;
29         }
30         somma
31     });
32
33     for p in producers {
34         p.join().unwrap();
35     }
36
37     transformer.join().unwrap();
38     let somma = consumer.join().unwrap();
39
40     println!("Somma dei quadrati: {}", somma);

```

41

```
}
```

Quesiti:

1. Il programma compila? Se no, si giustifichi la risposta. **(0,25 pt)**
2. Si descriva il comportamento del programma a runtime, spiegando in dettaglio il suo comportamento errato (es. deadlock / mancata terminazione dei cicli `for v in rx`). **(1,0 pt)**
3. Si corregga il programma per ottenere la corretta esecuzione che lo porti alla terminazione sicura con la stampa del valore acquisito a valle della terminazione del thread `consumer`. **(1,5 pt)**
4. Si indichi il valore numerico finale stampato dal programma corretto. **(0,25 pt)**

Domanda 3 (Teoria Asincronia & Runtime Tokio) — 3,0 punti

Si assuma nel file `Cargo.toml` la presenza della dipendenza: `tokio = { version = "1", features = ["full"] }`. Dato il seguente programma asincrono:

```
1 use tokio::time::{sleep, Duration};
2
3 async fn task(nome: &str, secondi: u64) {
4     println!("Inizio {}", nome);
5     sleep(Duration::from_secs(secondi)).await;
6     println!("Fine {}", nome);
7 }
8
9 #[tokio::main]
10 async fn main() {
11     let f1 = task("A", 2); // Domanda B
12     let f2 = task("B", 1);
13
14     println!("Future create");
15
16     f1.await;
17     f2.await;
18
19     println!("Main terminato");
20 }
```

Quesiti:

- A. Perché è necessario l'attributo `#[tokio::main]`? (0,5 pt)
- A. Che cosa produce la chiamata `task("A", 2)` (corrispondente all'istruzione commentata)? L'esecuzione del corpo della funzione inizia immediatamente? (0,5 pt)
- A. In quale ordine esatto vengono stampati i messaggi a video e qual è il tempo totale approssimativo di esecuzione dell'intero programma? (1,0 pt)
- A. Come si potrebbe modificare il `main` per ridurre il tempo totale di esecuzione, mantenendo comunque l'esecuzione concorrente dei due task? Si scriva il frammento di codice modificato e si indichi il nuovo tempo di esecuzione. (1,0 pt)

Parte II: Domanda di Programmazione (Valutazione: 6,0 punti)

Domanda 4 (Programming Question: Connection / Resource Pool)

Descrizione del Problema

Molti servizi reali non aprono una nuova connessione ogni volta che ne hanno bisogno: l'apertura è lenta e onerosa. Mantengono invece un insieme fisso di connessioni già pronte in un `**pool**`; chi deve lavorare ne preleva una, la usa e la restituisce, affinché torni disponibile per qualcun altro. È lo schema dei `*connection pool*` usati in molti contesti, come i database o i servizi web.

Si scriva in Rust una struttura che implementi il tratto generico `ResourcePool<T: Send>`,

che gestisce un insieme fisso di elementi riutilizzabili di tipo `T` e li concede in prestito esclusivo ai thread che ne fanno richiesta. Questi elementi devono essere racchiusi da un tipo che implementi il tratto generico `Resource<T: Send>`, che permette di accedere al valore dell'elemento concesso in prestito e che lo restituisce automaticamente al pool quando il valore esce dallo scope utilizzando il concetto di **RAII** (tramite il tratto `Drop`).

API Richiesta (Interfacce fornite)

```
1 pub trait Resource<T: Send> {
2     /// Restituisce un riferimento immutabile all'elemento contenuto.
3     fn get(&self) -> &T;
4 }
5
6 pub trait ResourcePool<T: Send> {
7     /// Numero totale di elementi gestiti dal pool.
8     fn capacity(&self) -> usize;
9
10    /// Preleva un elemento dal pool e lo consegna al chiamante. Se nessun
11    /// elemento e' disponibile, blocca il chiamante senza consumare cicli di CPU
12    /// finche' una risorsa non viene rilasciata.
13    fn acquire(&self) -> impl Resource<T>;
14
15    /// Variante con attesa limitata: come 'acquire', ma se non ottiene un
16    /// elemento entro 'timeout' rinuncia e restituisce 'None'.
17    /// L'attesa non deve consumare CPU.
18    fn acquire_timeout(&self, timeout: Duration) -> Option<impl Resource<T>>;
19 }
20
21 /// Costruttore principale per la creazione del pool a partire da un vettore di
22 risorse.
23 pub fn make_resource_pool<T: Send>(items: Vec<T>) -> impl ResourcePool<T> {
24     // ...
25 }
```

Requisiti di Implementazione

1. **Thread-Safety**: La struttura del pool deve essere thread-safe e condivisibile tra più thread contemporaneamente (`Arc`, `Mutex`, `Condvar`).
2. **Esclusività & RAII**: Ogni elemento del pool deve essere concesso ad al più un thread alla volta. Quando l'oggetto che implementa `Resource<T>` viene distrutto (uscita dallo scope), il valore di tipo `T` (con il proprio stato aggiornato) presente al suo interno deve tornare **automaticamente** nel pool tramite il tratto `Drop`, tornando subito disponibile per un'ulteriore richiesta.
3. **Assenza di Attesa Attiva**: Nessuna attesa attiva (*no busy-waiting*): i thread bloccati in attesa di una risorsa devono dormire sulla `Condvar` senza consumare cicli di CPU.